pontIA

MÁSTER INGENIERÍA DE DATOS

Consultas 6

Otras estructuras de datos - Recursos

Daniel Trabas

pontIA

Índice

Objetivo 3

Anexos 13

PontIA

Objetivo

El presente documento técnico sirve como apoyo y guía para el aprendizaje de tipos de datos avanzados, estructuras semiestructuradas, almacenamiento binario de alta capacidad y personalización de esquemas lógicos en PostgreSQL.

Todos los ejemplos prácticos se integran en el esquema de bookings en la base de datos demodb.

ARRAY: estructuras multidimensionales y desnormalización controlada

PostgreSQL permite definir columnas de una tabla como arrays multidimensionales de longitud variable, soportando casi cualquier tipo de dato base (numérico, caracteres o tipos personalizados). Un array es ideal para almacenar colecciones homogéneas donde el orden físico es semánticamente relevante o cuando una relación clásica de tipo 1:N añadiría una complejidad de uniones (JOINs) y latencia de red innecesaria en operaciones de sólo lectura masiva.

Algunos de los ejemplos dónde poder utilizar arrays son:

- **Catálogos de rutas con días operativos:** para almacenar de forma compacta qué días de la semana opera un tramo, evitando registrar tablas hijas de calendarios redundantes.
- **Preferencias y metadatos de usuario:** como almacenamiento de vectores de preferencias como comidas especiales a bordo, dimensiones de equipajes o idiomas sin requerir uniones complejas.
- **Análisis de series temporales sencillas:** para guardar valores de temperatura, lecturas de sensores o históricos mensuales de ventas de billetes por trimestre de forma ordenada.

PontIA

Ejemplos prácticos sobre demodb

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

Filtrado de rutas operativas en fin de Semana

En la tabla de rutas (bookings.routes), el campo days_of_week contiene un array de enteros que representa los días en que opera cada vuelo. El siguiente código localiza rutas operativas los fines de semana (sábado = 6, domingo = 7) utilizando el operador de solapamiento de arrays &&:

```
-- 1. Buscamos rutas que operen los fines de semana (sábado y domingo)
SELECT rt.route_no,
       rt.departure_airport,
       rt.arrival_airport,
       rt.days_of_week
FROM   bookings.routes rt
WHERE  rt.days_of_week && ARRAY[6, 7]
LIMIT 100;
  WHERE bk.book_ref = 'XJK300';
```

Creación de tabla de preferencias de pasajeros y población

Para documentar la alimentación y el equipaje de los pasajeros registrados, creamos la tabla bookings.passenger_preferences que almacena un array de texto para platos favoritos y un array numérico de 3 dimensiones para el equipaje [profundidad, ancho, alto]:

```
-- 2.Creación de la tabla de preferencias
CREATE TABLE bookings.passenger_preferences (
  passenger_id text PRIMARY KEY,
  preferred_meals text[] NOT NULL,
  baggage_dimensions numeric[] NOT NULL -- [profundidad, ancho, alto]
);
-- Inserción de 5 registros de prueba consistentes
INSERT INTO bookings.passenger_preferences (passenger_id, preferred_meals,
baggage_dimensions)
VALUES
('IN 7304486195142', ARRAY['Vegetarian', 'Gluten-Free'], ARRAY[55.0, 40.0,
20.0]),
('CN 7585878188590', '{Standard}', ARRAY[60.0, 45.0, 25.0]),
('FR 0491258157313', '{Kosher, Vegan}', ARRAY[50.0, 35.0, 15.0]),
('RU 9700594927910', ARRAY['Standard', 'Low-Sodium'], ARRAY[55.0, 40.0, 23.0]),
('CZ 2772651958166', ARRAY['Halal'], ARRAY[58.0, 42.0, 22.0]);
-- Validación de la inserción
SELECT tk.ticket_no,
       pp.passenger_id,
       tk.passenger_name,
       pp.preferred_meals,
       pp.baggage_dimensions
FROM   bookings.passenger_preferences pp
  INNER JOIN bookings.tickets tk ON pp.passenger_id = tk.passenger_id;
```

Auditoría de equipajes sobredimensionados mediante subíndices

Utilizamos el indexado físico de arrays en PostgreSQL (donde el primer elemento tiene el índice 1 por defecto) para auditar y alertar sobre pasajeros cuyo equipaje excede las dimensiones máximas permitidas de cabina (55 cm de profundidad, 40 cm de ancho, o 20 cm de alto):

```
-- 3. Buscamos pasajeros con dimensiones de equipaje que excedan los límites
permitidos (profundidad > 55 cm, ancho > 40 cm, alto > 20 cm)
SELECT pp.passenger_id,
       pp.baggage_dimensions[1] AS profundidad_cm,
       pp.baggage_dimensions[2] AS ancho_cm,
       pp.baggage_dimensions[3] AS alto_cm
FROM bookings.passenger_preferences pp
WHERE ( pp.baggage_dimensions[1] > 55.0
   OR   pp.baggage_dimensions[2] > 40.0
   OR   pp.baggage_dimensions[3] > 20.0 );
```

JSON y JSONB: Persistencia semiestructurada y alta velocidad

PostgreSQL ofrece dos tipos de datos avanzados para persistir esquemas lógicos flexibles e interactuar con documentos JSON. Aunque ambos validan la sintaxis interna de las cadenas de texto JSON ingresadas, sus implementaciones físicas difieren drásticamente:

Característica	Tipo JSON (clásico)	Tipo JSONB (descompuesto)
Almacenamiento	Guarda el texto exacto original con espacios y formato	Guarda los datos en un formato binario
Orden de llaves	Preserva el orden original de inserción de las llaves.	No preserva el orden de llaves (lo reescribe)
Duplicados	Conserva llaves duplicadas dentro del documento.	Elimina llaves duplicadas (prevalece la última).
Indexación	No permite indexación directa; requiere índices en expresiones.	Soporta indexación de alta velocidad (GIN, GIST).
Velocidad de Escritura	Rápido al insertar, ya que no procesa de forma binaria.	Ligeramente más costoso debido al preprocesamiento.
Velocidad de Lectura	Lento, debe analizar y parsear el texto cada vez.	Extremadamente rápido al acceder a elementos concretos.

Ejemplos prácticos sobre demodb

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

Extracción de datos multilingües en modelos de aviones y ciudades

En la base de datos, los nombres de aeropuertos, ciudades y modelos de aviones se guardan como objetos JSONB para dar soporte multi-idioma (traducciones en inglés, ruso, etc.). El siguiente script utiliza el operador de extracción como texto →› para extraer las etiquetas correspondientes:

```
-- 1. Extraemos el código de aeropuerto, la ciudad y el modelo de avión en
varios idiomas utilizando JSONB
SELECT aid.airport_code,
       aid.city->>'en' AS ciudad_ingles,
       aid.city->>'ru' AS ciudad_ruso,
       apd.model->>'en' AS modelo_avion
FROM   bookings.airports_data aid
  CROSS JOIN bookings.airplanes_data apd
LIMIT 100;
```

Filtrado por contención de documentos JSONB

Para realizar un filtrado óptimo que pueda aprovechar los índices GIN, utilizamos el operador de contención @› para encontrar únicamente aquellos aeropuertos ubicados en la ciudad de 'London' de forma segura e independiente del orden de las llaves:

```
-- 2. Filtra aeropuertos cuya clave de ciudad contenga la traducción exacta
SELECT aid.airport_code,
       aid.airport_name->>'en' AS nombre_aeropuerto_en,
       aid.timezone
FROM   bookings.airports_data aid
WHERE  aid.city @> '{"en": "London"}'::jsonb;
```

Tabla de metadatos de vuelos y búsqueda sobre atributos Internos

Creamos una tabla auxiliar para almacenar bitácoras dinámicas de vuelos (datos climáticos, pilotos asignados, alertas de catering) y realizamos una búsqueda para identificar vuelos que sufrieron vientos superiores a 20 nudos:

```
-- 3. Búsqueda de vuelos que tuvieron vientos superiores a 20 nudos
-- Crear tabla de metadatos de vuelo
CREATE TABLE bookings.flight_metadata (
  flight_id integer PRIMARY KEY REFERENCES bookings.flights(flight_id)
  ON UPDATE CASCADE
  ON DELETE CASCADE,
  details jsonb NOT NULL
);
-- Insertar metadatos variados en formato JSONB
INSERT INTO bookings.flight_metadata (flight_id, details) VALUES
(1234, '{"pilot": "Capt. Smith", "weather": {"temp": 15, "wind": 12},
"delay_reason": null}'),
(5678, '{"pilot": "Capt. Jones", "weather": {"temp": -5, "wind": 25},
"delay_reason": "De-icing required"}'),
(9012, '{"pilot": "Capt. Taylor", "weather": {"temp": 20, "wind": 5},
"special_service": ["VIP onboard"]}')
-- Verificación de la inserción
SELECT *
FROM   bookings.flight_metadata
WHERE  flight_id IN (1234, 5678, 9012);
-- Consulta para buscar vientos críticos superiores a 20 nudos
SELECT fm.flight_id,
       fm.details->'weather'->>'wind' AS velocidad_viento,
       fm.details->>'pilot' AS piloto
FROM   bookings.flight_metadata fm
WHERE  (fm.details->'weather'->>'wind')::integer > 20;
```

BLOPs (BYTEA, pg_largeobject): Gestión de contenidos binarios

PostgreSQL maneja datos binarios de gran tamaño mediante dos alternativas con filosofías de almacenamiento y arquitecturas de acceso físicamente independientes:

Criterio	Tipo BYTEA	Subsistema pg_largeobject (LOB)
Almacenamiento	Se guarda en la fila del registro. Si excede 2KB, se comprime y guarda vía TOAST.	Se fragmenta en chunks de 2KB almacenados en la tabla del sistema pg_largeobject.
Tamaño Máximo	Hasta 1 Gigabyte (1GB) por celda.	Soporta archivos binarios masivos de hasta 4 Terabytes (4TB).
Acceso y Lectura	Se lee de golpe en memoria RAM. Es ineficiente para archivos pesados.	Soporta lectura y escritura parcial por streaming (random access, seek).
Representación	Se almacena como una columna ordinaria en la tabla física.	La tabla contiene un identificador numérico (OID) que apunta al LOB.
Gestión	Se gestiona con comandos estándar DML (INSERT, SELECT, UPDATE).	Requiere el uso de funciones especiales de sistema (lo_create, lo_open).

Ejemplos prácticos sobre demodb

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

Almacenamiento de boarding passes en formato BYTEA

Almacenar archivos PDF pequeños (menos de 2MB) correspondientes a tarjetas de embarque utilizando el tipo BYTEA de forma enlazada a las tablas de tickets y pases de abordar de la aerolínea:

PontIA

```sql
-- 1. Almacenamiento de tarjetas de embarque pdf en una tabla
-- Tabla para guardar el archivo binario del boarding pass
CREATE TABLE bookings.boarding_passes_pdf (
  ticket_no varchar(13) NOT NULL,
  flight_id integer NOT NULL,
  pdf_data bytea NOT NULL,
  PRIMARY KEY (ticket_no, flight_id),
  FOREIGN KEY (ticket_no) REFERENCES bookings.tickets (ticket_no)
  ON UPDATE CASCADE
  ON DELETE CASCADE,
  FOREIGN KEY (flight_id) REFERENCES bookings.flights (flight_id)
  ON UPDATE CASCADE
  ON DELETE CASCADE
);
-- Verificamos la tabla
SELECT * FROM bookings.boarding_passes_pdf;

-- Inserción de un registro binario usando el formato hexadecimal (\\x...)
INSERT INTO bookings.boarding_passes_pdf (ticket_no, flight_id, pdf_data)
VALUES
  ( '0005437176449', 27671,
     pg_read_binary_file('C:\localdb\_scripts_\tempfiles\boarding-pass-5437176449.pdf')
  ),
  ( '0005437176467', 27671,
     pg_read_binary_file('C:\localdb\_scripts_\tempfiles\boarding-pass-5437176467.pdf')
  ),
  ( '0005437176554', 27671,
     pg_read_binary_file('C:\localdb\_scripts_\tempfiles\boarding-pass-5437176554.pdf')
  ),
  ( '0005437176590', 27671,
     pg_read_binary_file('C:\localdb\_scripts_\tempfiles\boarding-pass-5437176590.pdf')
  ),
  ( '0005437176975', 27671,
     pg_read_binary_file('C:\localdb\_scripts_\tempfiles\boarding-pass-5437176975.pdf')
  );
-- Verificamos la inserción de todos los datos
SELECT * FROM bookings.boarding_passes_pdf;

-- Validación de la inserción
SELECT ticket_no,
       flight_id,
       pdf_data,
       length(pdf_data) AS tamaño_bytes
FROM   bookings.boarding_passes_pdf
WHERE  ticket_no = '0005437176449';
```

Asociación de manuales de mantenimiento con OIDs de Large Objects

Para gestionar planos técnicos o manuales pesados de mantenimiento de aeronaves, creamos un registro de referencia asado en OIDs vinculados al catálogo central de objetos grandes de PostgreSQL:

```
-- 2. Almacenamiento de objetos grandes o pesados como planos de aviones
-- Tabla de manuales de mantenimiento asociados a modelos de la flota
CREATE TABLE bookings.airplane_documents (
  document_id serial PRIMARY KEY,
  document_name text NOT NULL,
  airplane_code bpchar(3) NOT NULL REFERENCES bookings.airplanes_data(airplane_code)
  ON UPDATE CASCADE
  ON DELETE CASCADE,
  manual_data oid NOT NULL -- OID que referencia a pg_largeobject
);
-- Validamos la creación de la tabla
SELECT * FROM bookings.airplane_documents;
-- Creación e inserción de un objeto grande en una transacción segura
BEGIN;
  INSERT INTO bookings.airplane_documents (document_name, airplane_code, manual_data)
  VALUES
('777-200-200ER-300_Rev_E.pdf','77W',lo_import('C:/localdb/_scripts_/tempfiles/777-200-2
00ER-300_Rev_E.pdf'));
COMMIT;
-- Validamos la inserción en la tabla
SELECT * FROM bookings.airplane_documents;
```

Creación de históricos de cajas negras mediante SELECT INTO

Utilizamos el comando DDL SELECT INTO para consolidar rápidamente un histórico de vuelos completados con éxito, inyectando un marcador binario de tipo BYTEA que albergará los registros crudos de vuelo:

PontIA

```
-- 3. Inserción de BYTEA por medio de SELECT INTO y datos dinámicos
-- Crear tabla de forma rápida con datos históricos y marcador BYTEA para registros con
JSON de valores dinámicos
SELECT DISTINCT fl.flight_id,
         fl.route_no,
         ai.airplane_code,
         fl.status,
-- Código dinámico de inserción
DECODE ('{
  "flight_id": "'||fl.flight_id||'",
  "route_no": "'||fl.route_no||'",
  "airplane_code": "'||ai.airplane_code||'",
  "flight_date": "'||NOW()||'",
  "events": [
     {
       "event_id": "EVT-'||FLOOR(RANDOM() * (5000 - 1000 + 1) + 1000)::int||'",
       "event_date": "'||CURRENT_TIMESTAMP||'",
       "event_data": [
         { "sensor": "altitude", "value": '||FLOOR(RANDOM() * (37000 - 10000 + 1) +
10000)::int||', "unit": "ft" },
         { "sensor": "cruising_speed", "value": '||FLOOR(RANDOM() * (845 - 360 + 1) +
360)::int||', "unit": "knots" },
         { "sensor": "stabilizers_status", "value": "'||(ARRAY['high', 'top', 'neutral',
'down', 'locked'])[floor(random() * 5) + 1]||'", "unit": "state" },
         { "sensor": "temperature", "value": '||TO_CHAR((RANDOM() * (-40.0 - (-45.0)) +
(-45.0)),'99.99')||', "unit": "celsius" },
         { "sensor": "fuel_remaining", "value": '||FLOOR(RANDOM() * (24000 - 8000 + 1) +
8000)::int||', "unit": "kg" }
       ]
     },
     {
       "event_id": "EVT-'||FLOOR(RANDOM() * (5000 - 1000 + 1) + 1000)::int||'",
       "event_date": "'||CURRENT_TIMESTAMP||'",
       "event_data": [
         { "sensor": "altitude", "value": '||FLOOR(RANDOM() * (37000 - 10000 + 1) +
10000)::int||', "unit": "ft" },
         { "sensor": "fuel_remaining", "value": '||FLOOR(RANDOM() * (24000 - 8000 + 1) +
8000)::int||', "unit": "kg" }
       ]
     }
  ]
}','escape')::bytea AS blackbox_log -- Campo BYTEA marcador de posición
INTO     bookings.flight_blackbox_logs
FROM     bookings.flights fl
  INNER JOIN bookings.routes ro ON fl.route_no = ro.route_no
  INNER JOIN bookings.airplanes ai ON ai.airplane_code = ro.airplane_code
WHERE    fl.status = 'Arrived'
GROUP BY fl.flight_id,
          fl.route_no,
          ai.airplane_code,
         fl.status
LIMIT 1000;
```

PontIA

```
-- Establecer la clave primaria correspondiente para garantizar el indexado
ALTER TABLE bookings.flight_blackbox_logs ADD PRIMARY KEY (flight_id,airplane_code);
-- Validar la creación de la tabla
SELECT * FROM bookings.flight_blackbox_logs;
```

Nota de seguridad sobre lo_import()

La función SQL lo_import() se ejecuta con los privilegios del usuario del sistema operativo que corre el motor de bases de datos (postgres). Por motivos de seguridad, su uso directo en consultas SQL está restringido por defecto a los superusuarios del motor o a aquellos roles a los que se les haya concedido explícitamente el rol del sistema pg_read_server_files

Nota sobre borrado de binarios en pg_largeobject

Si se borra un registro de la tabla de negocio (DELETE FROM bookings.airplane_documents WHERE airplane_code = '77W';), el archivo binario en pg_largeobject NO se elimina automáticamente. El OID quedará huérfano en el catálogo del sistema consumiendo espacio en disco.

Para evitar esto, hay que hacer un borrado físico de un objeto grande en PostgreSQL usando la función del servidor lo_unlink(oid):

```
-- Borrado físico y definitivo del almacenamiento en pg_largeobject
SELECT lo_unlink(<identificador de objeto>);
```

CREATE TYPE: Extensibilidad y personalización de tipos de datos

Tipos compuestos y enumerados (ENUM)

PostgreSQL es un motor relacional de objetos, lo que significa que el usuario puede definir sus propios tipos de datos personalizados mediante la cláusula CREATE TYPE

Los dos tipos personalizados de uso más común en producción son:

- **Tipos Enumerados (ENUM):** representan un conjunto estático y ordenado de valores discretos. PostgreSQL almacena físicamente estos valores como identificadores numéricos de 4 bytes, lo que ahorra un espacio inmenso y optimiza los filtros frente al texto plano tradicional.
- **Tipos Compuestos (Composite):** permiten agrupar múltiples atributos (con sus respectivos tipos de datos) en una sola estructura lógica. Se comportan como un registro o tupla que puede ser utilizado como el tipo de datos de una sola columna ordinaria.

Ejemplos prácticos sobre demodb

(Nota: Asumimos las entidades clásicas de demodb: bookings, airports, airplanes, etc.).

Clasificación de carga y prioridad mediante ENUM

Definimos un tipo ENUM para representar la prioridad de envío de mercancías y lo implementamos en un manifiesto de carga aérea. Al ser un ENUM ordenado, PostgreSQL nos permite realizar ordenaciones y filtros utilizando operadores matemáticos tradicionales:

```
-- 1. Creación de un tipo numerado propio para nuestro array de prioridad de
carga
/*
-- Solo para DOCENTE:
DROP TABLE IF EXISTS cargo_manifest;
DROP TYPE IF EXISTS bookings.cargo_priority;
*/
-- Crear el tipo enumerado de prioridad de carga
CREATE TYPE bookings.cargo_priority AS ENUM ('Low', 'Medium', 'High',
'Critical');
-- Tabla de manifiesto de carga
CREATE TABLE bookings.cargo_manifest (
  manifest_id serial PRIMARY KEY,
  flight_id integer NOT NULL REFERENCES bookings.flights(flight_id)
  ON UPDATE CASCADE
  ON DELETE CASCADE,
  weight_kg numeric(8,2) NOT NULL,
  priority bookings.cargo_priority NOT NULL
);
-- Inserción de registros de prueba
INSERT INTO bookings.cargo_manifest (flight_id, weight_kg, priority)
SELECT fl.flight_id,
       (RANDOM() * (15000 - 2500) + 2500)::NUMERIC(8,2) AS weight_kg,
       (ARRAY['Low', 'Medium', 'High', 'Critical'])[floor(random() * 4) +
1]::bookings.cargo_priority AS priority
FROM   bookings.flights fl
  INNER JOIN bookings.routes ro ON fl.route_no = ro.route_no
  INNER JOIN bookings.airplanes ai ON ai.airplane_code = ro.airplane_code
WHERE  fl.status = 'Arrived'
GROUP BY fl.flight_id
LIMIT 100;
-- Consulta para encontrar cargas que requieren especial atención (High o
Critical)
SELECT cm.manifest_id,
       cm.weight_kg,
       cm.priority
FROM   bookings.cargo_manifest cm
WHERE  cm.priority >= 'High'; -- El motor evalúa el orden de definición del
ENUM
```

PONIA

Dimensiones de carga 3D utilizando un tipo compuesto

Definimos un tipo compuesto para almacenar las dimensiones físicas tridimensionales de los bultos en cabina y mostramos cómo realizar inserciones y cómo acceder a los elementos internos usando la notación parentética :

```sql
-- 2. Creación de un tipo para soporte de datos 3D
-- Crear tipo compuesto de tres dimensiones
CREATE TYPE bookings.dimension_3d AS (
  depth numeric(5,2),
  width numeric(5,2),
  height numeric(5,2)
);
-- Tabla de dimensiones de carga
CREATE TABLE bookings.cargo_dimensions (
  cargo_id serial PRIMARY KEY,
  manifest_id integer NOT NULL REFERENCES bookings.cargo_manifest(manifest_id)
  ON UPDATE CASCADE
  ON DELETE CASCADE,
  dimensions bookings.dimension_3d NOT NULL
);
-- Inserción utilizando el constructor de fila ROW() o casteo
INSERT INTO bookings.cargo_dimensions (manifest_id, dimensions)
VALUES (1, ROW(120.0, 80.0, 50.0)::bookings.dimension_3d),
       (2, ROW(58.0, 71.0, 35.0)::bookings.dimension_3d),
       (3, ROW(130.0, 40.0, 40.0)::bookings.dimension_3d),
       (4, ROW(60.0, 120.0, 20.0)::bookings.dimension_3d),
       (5, ROW(180.0, 180.0, 80.0)::bookings.dimension_3d),
       (6, ROW(100.0, 100.0, 100.0)::bookings.dimension_3d),
       (7, ROW(94.0, 36.0, 56.0)::bookings.dimension_3d),
       (8, ROW(75.0, 45.0, 75.0)::bookings.dimension_3d);
-- Consulta técnica: Acceso a un atributo interno del tipo compuesto
-- NOTA: Se requiere rodear el campo compuesto de paréntesis antes del punto
SELECT cd.cargo_id,
       (cd.dimensions).depth AS profundidad_bulto,
       (cd.dimensions).width AS ancho_bulto,
       (cd.dimensions).height AS alto_bulto
FROM   bookings.cargo_dimensions cd
WHERE  (cd.dimensions).width > 90.0
  AND  (cd.dimensions).depth > 80.0;
```

Anexos

Tipos de datos y declaración de arrays

A continuación se presenta un cuadro técnico con ejemplos de los principales tipos de datos que admiten arrays y cómo definirlos mediante la sintaxis nativa de PostgreSQL y la sintaxis estándar de SQL ANSI:

Categoría de Tipo de Datos	Tipo Base de Ejemplo	Declaración (Sintaxis PostgreSQL)	Declaración (Sintaxis Estándar SQL)	Comportamiento / Detalles Técnicos
Numéricos Integrados	integer	integer[]	integer ARRAY	Almacena colecciones de números enteros
Punto Flotante	double precision	double precision[]	double precision ARRAY	Almacena valores numéricos con decimales de precisión variable.
Cadenas de Texto	text	text[] o text[][]	text ARRAY	Útil para almacenar listas de caracteres o cadenas. Admite múltiples dimensiones concatenadas.
Tipos de Fecha y Hora	timestamp	timestamp[]	timestamp ARRAY	Almacena un conjunto de fechas con hora sin zona horaria.
Tipos Lógicos	boolean	boolean[]	boolean ARRAY	Almacena valores de tipo booleano (true, false, NULL).
Tipos Enumerados (ENUM)	rainbow (creado por usuario)	rainbow[]	rainbow ARRAY	Almacena un conjunto ordenado de etiquetas predefinidas constantes.
Tipos Compuestos (Composite)	inventory_item (usuario)	inventory_item[]	inventory_item ARRAY	Almacena colecciones de tuplas complejas estructuradas con múltiples campos internos.
Tipos de Rango (Range)	tsrange (rango de tiempos)	tsrange[]	tsrange ARRAY	Almacena conjuntos de rangos de datos ordenados.
Tipos de Dominio (Domain)	mytext (usuario)	mytext[]	mytext ARRAY	Permite coleccionar tipos personalizados que contienen restricciones de validación adicionales.

Filtros GIN, GIST de los JSONB en PostgreSQL

En PostgreSQL, el soporte para almacenar y consultar datos semiestructurados en columnas de tipo jsonb (JSON binario descompuesto) es una de sus características más potentes. Para acelerar las búsquedas dentro de estos documentos sin tener que recorrer todo el texto linealmente, el motor permite el uso de índices avanzados.

Índices GIN (Generalized Inverted Index) en JSONB

El índice GIN es un índice invertido. Está diseñado específicamente para optimizar búsquedas en tipos de datos que contienen múltiples componentes o elementos internos dentro de un solo registro físico, como es el caso de los arrays y los documentos jsonb

En lugar de indexar la celda completa, un índice GIN extrae los elementos individuales (llaves y valores) del documento JSONB y crea entradas separadas apuntando al registro correspondiente. Esto permite realizar búsquedas sumamente rápidas de claves o parejas clave-valor dentro de colecciones masivas de JSONB

PostgreSQL ofrece dos clases de operadores GIN para configurar de forma personalizada el comportamiento del índice según las necesidades de tu aplicación:

Clase de operadores jsonb_ops (Predeterminada)

Es la clase de operadores que PostgreSQL aplica por defecto si creas un índice GIN convencional sobre una columna JSONB (por ejemplo: `CREATE INDEX ON tabla USING GIN (columna_jsonb)`)

.

- **Operadores soportados:** es altamente flexible y da soporte a los operadores de existencia de claves (?, ?|, ?&), al operador de contención lógica (@>) y a los operadores de coincidencia con rutas SQL/JSON (@? y @@)
- **Funcionamiento interno:** genera entradas de índice independientes para cada llave y cada valor contenido en el documento. Por ejemplo, si tienes el documento {"foo": {"bar": "baz"}}, el índice jsonb_ops creará tres ítems individuales en el diccionario físico: uno para la clave foo, otro para la clave bar y otro para el valor baz
- **Pros y Contras:** su gran beneficio es la versatilidad, ya que te permite preguntar de manera directa si una clave específica existe en cualquier parte del JSON. El inconveniente es que el índice tiende a ser físicamente más grande en disco y la velocidad de inserción o actualización de filas puede verse afectada debido a la cantidad de sub-elementos que debe procesar el motor

Clase de operadores jsonb_path_ops (Optimizado)

Esta clase de operadores no predeterminada se debe declarar de forma explícita al construir el índice (ej: `CREATE INDEX ON tabla USING GIN (columna_jsonb jsonb_path_ops)`)

- **Operadores soportados:** no es compatible con los operadores de existencia de claves (?, ?|, ?&). Sin embargo, soporta de forma nativa las consultas de contención (@>) y coincidencias de rutas (@? y @@)
- **Funcionamiento interno:** en lugar de indexar llaves y valores por separado, genera entradas únicamente para los valores del documento, aplicando una función hash que combina el valor y toda la ruta de llaves que conduce a él. Tomando el mismo ejemplo de {"foo": {"bar": "baz"}}, se creará una sola entrada en el índice basada en el hash combinado de la ruta lógica foo → bar → baz
- **Pros y Contras:** es un índice considerablemente más pequeño en disco y mucho más rápido al ejecutar consultas de contención selectivas, sobre todo cuando se busca información bajo llaves muy repetitivas. Su desventaja es que no es útil para verificar la mera existencia de llaves intermedias y no genera entradas para estructuras JSON vacías como {"a": {}}

Índices GiST (Generalized Search Tree) en PostgreSQL

A diferencia de GIN, un índice GiST no representa un tipo único de ordenamiento, sino que es una infraestructura altamente extensible sobre la cual se pueden implementar diversas estrategias personalizadas de indexación. Aunque en PostgreSQL el estándar de la industria utiliza GIN para la flexibilidad interna de documentos jsonb, la arquitectura GiST destaca por su propio comportamiento y aplicaciones en el motor.

- **Estructura pérdida (Lossy):** una de las propiedades clave de GiST descritas en la documentación es que puede ser un índice con pérdidas o lossy. Esto significa que el índice puede producir falsas coincidencias (false matches) debido a que comprime y representa los documentos o elementos en firmas hash de longitud fija. Cuando se realiza una consulta, Pos-

PontIA

tgreSQL utiliza el índice para aislar los candidatos y luego accede automáticamente a la fila física de la tabla (el heap) para realizar una re-comprobación estricta de la condición.

- **Soporte de Index-Only Scans:** en PostgreSQL, los índices GiST admiten búsquedas directas en el índice sin tocar el disco (index-only scans) para ciertas clases de operadores, siempre y cuando la estrategia pueda reconstruir el valor original. Los índices GIN no admiten index-only scans porque solo almacenan fragmentos o componentes de los datos originales, lo que hace obligatorio visitar siempre la tabla real.

.

- **Aplicaciones principales en PostgreSQL:**
  - **Tipos Geométricos y Espaciales:** es el estándar predeterminado para indexar puntos, líneas, cajas de coordenadas y polígonos, soportando operadores de solapamiento (&&), contención (@>) y búsquedas de vecinos más cercanos (<->)
  - **Rangos y Multirangos:** permite acelerar de manera sobresaliente las consultas de intervalos temporales o numéricos en restricciones de exclusión de reservas (ej. evitar que se reserven habitaciones en horarios duplicados)

Resumen para la toma de decisiones sobre qué filtro usar

Para almacenar y consultar colecciones de documentos flexibles en formato JSONB bajo PostgreSQL:

1. Usa un índice GIN con jsonb_ops si tu aplicación realiza búsquedas variadas, incluyendo comprobar si ciertas llaves existen dinámicamente en los documentos
2. Usa un índice GIN con jsonb_path_ops si requieres optimizar el espacio en disco y tus consultas se basan estrictamente en verificar si los documentos contienen valores o estructuras internas específicas (@>)
3. Si requieres realizar comprobaciones estrictas de igualdad completa entre documentos JSON enteros, se recomienda recurrir a índices tradicionales de tipo B-Tree o Hash

PontIA